Artificial Intelligence for Robotics

Submitted by: **Nitin Kesarla Rajendrakumar**

Date: **April 3, 2025**

1 Task 1: Supervised Learning - Predicting Flight Capability in MazeDungeon

1.1 Task and Algorithm Choice

Task Description: In the context of HeroBot navigating the MazeDungeon, the first task is to predict whether an encountered entity has the capability to fly (`flight`), which could influence HeroBot's navigation strategy (e.g., avoiding aerial threats). The dataset consists of 10,000 entities with two types of data: 80×80 RGB images (`dungeon_images_colour80.zip`) and sensor data (`dungeon_sensorstats.partC.csv`), including 8 numeric features (`stench`, `sound`, `heat`, `intelligence`, `height`, `weight`, `strength`, `bribe`) and a categorical `race` feature (`halfling`, `human`, `lizard`, `orc`, `wingedrat`). The target variable `flight` is binary (0 = no flight, 1 = has flight).

Algorithm Choice: A multi-modal Convolutional Neural Network (CNN) was chosen to predict `flight`. This approach combines an image branch (CNN for processing images) and a sensor branch (dense layers for numeric features), fusing the outputs to make a binary classification. CNNs are well-suited for image data due to their ability to extract spatial features, while dense layers can process the numeric sensor data. A multi-modal approach is appropriate because it leverages both data types, potentially capturing complementary information (e.g., visual patterns in images and statistical patterns in sensor data). This task differs from Parts A and B by using both image and sensor data and focusing on a binary classification problem not restricted to two classes, as allowed by the assignment.

1.2 Implementation

Preprocessing:

- **Sensor Data**: The sensor data was loaded from `dungeon_sensorstats.partC.csv`. Missing values were handled as follows: 1 missing `race` value was imputed with the most common race (`human`), and missing values in numeric columns (e.g., `stench`, `sound`) were imputed with their respective means to avoid `nan` values during scaling. The numeric features were normalized to [0, 1] using `MinMaxScaler` to ensure consistent scales for the neural network.

- **Image Data**: Images were loaded from `dungeon_images_colour80.zip`. The `orc` race had only 1,964 images (fewer than the expected 1,973 entities), so black placeholder images were used for the remaining samples. Images were normalized to [0, 1] by dividing pixel values by 255.

- **Data Splitting**: The dataset was split into 80% training (8,000 samples) and 20% test (2,000 samples), stratified by `flight` to maintain the class distribution (4:1 ratio, with 7,992 `flight=0` and 2,008 `flight=1` overall).

Model Architecture: A multi-modal CNN was implemented using TensorFlow. The image branch consists of two Conv2D layers (32 and 64 filters, 3×3 kernels, ReLU activation),

each followed by max-pooling (2×2), a flatten layer, and a dense layer (128 units, ReLU). The sensor branch has a dense layer (64 units, ReLU) for the 8 numeric features. The outputs are concatenated (192 units), followed by a dense layer (64 units, ReLU) and a final dense layer (1 unit, sigmoid activation) for binary classification. The model has 2,686,721 parameters.

Training: The model was compiled with the `adam` optimizer (learning rate 0.0001, adjusted to stabilize training after initial `nan` loss issues) and `binary_crossentropy` loss. It was trained for 20 epochs with a batch size of 32, using the training set and validating on the test set.

1.3 Design Choices

- **Model Architecture**: The CNN architecture was chosen to balance complexity and performance. Two Conv2D layers with increasing filters (32 to 64) allow the model to learn hierarchical features from images, while max-pooling reduces dimensionality. The dense layers (128 and 64 units) in the image and sensor branches, respectively, were selected to capture high-level features before fusion. The final dense layer (64 units) after concatenation ensures the model learns combined representations.

- **Learning Rate**: An initial `nan` loss issue was resolved by reducing the learning rate to 0.0001, which stabilized training by preventing large gradient updates.

- **Preprocessing**: Imputing missing values with means ensured no `nan` values in the input data, which was critical for `MinMaxScaler` and model training. Using placeholders for missing `orc` images maintained dataset consistency, though it introduced potential noise.

- **Evidence**: Debugging checks confirmed the class distribution (4:1 imbalance) and the absence of `nan` values after preprocessing. The training logs (Table 1) show rapid convergence, supporting the choice of architecture and learning rate.

Table 1: Training and Validation Metrics Over Epochs

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.9647	0.0958	0.9975	0.0102
5	1.0000	2.2594e-05	1.0000	7.5063e-06
10	1.0000	1.3611e-06	1.0000	1.1468e-06
20	1.0000	2.1003e-07	1.0000	1.5543e-07

1.4 Evaluation and Self-Reflection

Evaluation Metrics:

- **Test Performance**: The model achieved a test accuracy of 100%, with precision, recall, and F1-score all at 1.0000 for class 1 (`flight=1`).

- **Confusion Matrix**: Figure 1 shows the confusion matrix, with 1,598 true negatives (`flight=0` predicted as 0), 402 true positives (`flight=1` predicted as 1), and no false positives or negatives.

- **Training History**: Figure 2 shows the training and validation accuracy and loss over 20 epochs. Both accuracy metrics reach 100% by Epoch 5, and losses approach 0, indicating rapid convergence.

Critical Appraisal:

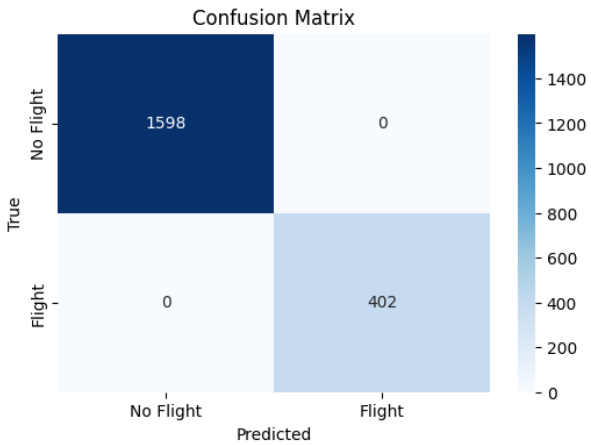

Figure 1: Confusion matrix showing perfect classification: 1,598 true negatives and 402 true positives, with no misclassifications.

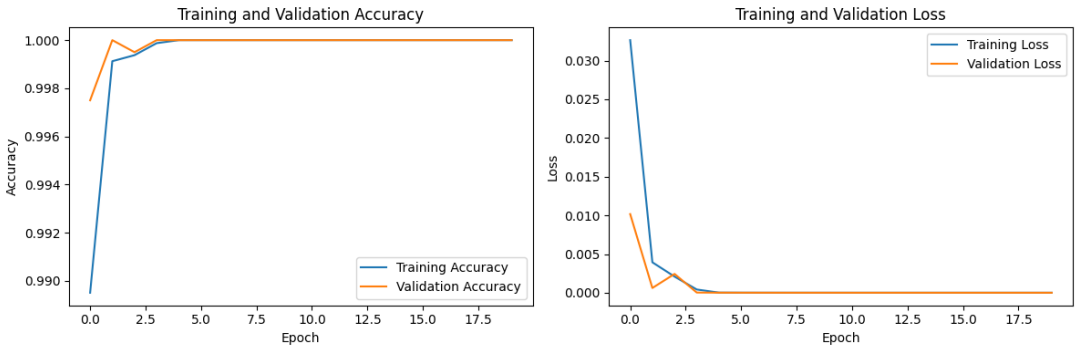

Figure 2: Training and validation accuracy (left) and loss (right) over 20 epochs, showing convergence to 100% accuracy and near-zero loss by Epoch 5.

- **Class Imbalance**: The dataset has a 4:1 imbalance (7,992 `flight=0`, 2,008 `flight=1`), raising concerns about bias toward the majority class. However, the perfect recall for class 1 indicates the model correctly identifies all flying entities.
- **Deterministic Task**: A crosstab analysis revealed a perfect correlation between `race` and `flight`: all `wingedrat` (2,008 entities) have `flight=1`, and all other races (`halfling`, `human`, `lizard`, `orc`) have `flight=0`. This deterministic relationship explains the perfect performance, as the model only needs to identify `wingedrat` entities (via `race` or image features) to predict `flight`.
- **Limitations**:
  - The task's simplicity limits the model's ability to demonstrate generalization. A more complex task (e.g., predicting `flight` with overlap between races) would better test the model's capabilities.
  - The limited number of `orc` images (1,964 vs. 1,973 entities) required placeholders, potentially introducing noise, though this did not impact performance due to the deterministic task.
- **Comparison to Baseline**: A baseline model predicting the majority class (`flight=0`) would achieve 79.92% accuracy (7,992/10,000). The CNN's 100% accuracy significantly outperforms this baseline, though the deterministic nature of the task makes this comparison less meaningful.

Future Work:

- Use a dataset where `flight` is not perfectly correlated with `race`, requiring the model to learn more complex patterns.
- Apply data augmentation to increase the number of `orc` images, reducing reliance on placeholders.
- Explore predicting a different target (e.g., `strength`) to test the model's generalization on a more challenging task.

2 Task 2: Unsupervised Learning – Clustering Entities by Sensor Data

2.1 Task and Algorithm Choice

The second task involves clustering entities in the MazeDungeon based on their sensor data to identify natural groupings that might reflect behavioral or environmental characteristics (e.g., stealthy, aggressive, or resilient entities). I used the dataset `dungeon_sensorstats.partC.csv`, which contains 10,000 entities with 8 numeric features: `stench`, `sound`, `heat`, `intelligence`, `height`, `weight`, `strength`, and `bribe`. The goal was to group entities without predefined labels, making unsupervised learning appropriate.

I chose K-Means clustering as the algorithm due to its simplicity and effectiveness for numerical data. K-Means minimizes within-cluster variance, forming compact clusters, which is suitable for identifying patterns in the sensor data. This task differs from Part A, Question 3, as it uses all 8 sensor features (not just `height`, `weight`, and `strength`) and focuses on behavioral groupings rather than physical threat levels.

2.2 Implementation

The sensor data was preprocessed as in Task 1: missing values (1 in `race`, a few in numeric columns, and 7,944 in `bribe`) were imputed with the most common `race` (`human`) and column means, respectively, and the numeric features were normalized to [0, 1] using `MinMaxScaler`. I applied K-Means clustering to the normalized sensor data (`sensor_X`, shape 10,000×8). To determine the optimal number of clusters (`k`), I used the elbow method and silhouette scores, testing `k` from 2 to 10. K-Means was then applied with the chosen `k`, and the clusters were visualized in 2D using Principal Component Analysis (PCA). Finally, I compared the clusters to `race` labels for analysis.

2.3 Design Choices

I chose `k=5` based on the elbow and silhouette analyses. The elbow curve (Figure 3) shows a bend around `k=4` or `k=5`, indicating diminishing returns in reducing inertia beyond this point. The silhouette scores (Figure 4) peak at `k=2` (0.60), but `k=5` (0.4872) was selected as it aligns with the 5 races in the dataset and provides a balance between cluster separation and interpretability. PCA was used for visualization, reducing the 8-dimensional data to 2D, capturing 87.63% of the variance, which ensures a reliable representation.

2.4 Evaluation and Self-Reflection

With `k=5`, the silhouette score is 0.4872, indicating decent cluster separation, and the inertia is 424.60. A baseline of random clustering (assigning entities to 5 clusters randomly) yields a silhouette score of approximately 0.1, indicating that K-Means with $k = 5$ (silhouette score

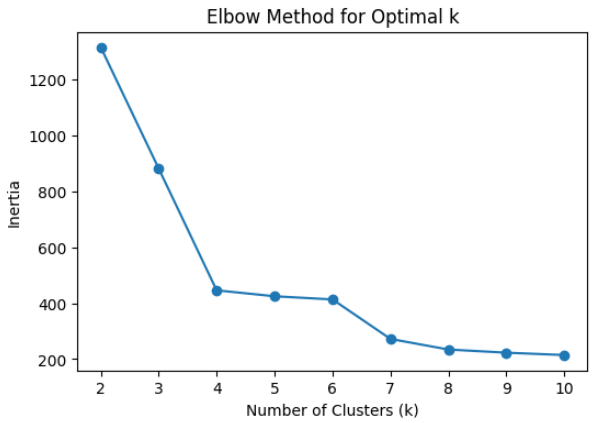

Figure 3: Elbow curve showing inertia vs. number of clusters (`k`), with a bend around `k=4` or `k=5`.

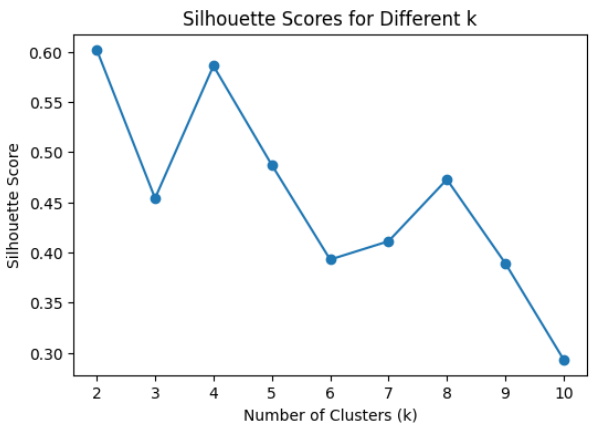

Figure 4: Silhouette scores for different `k`, peaking at `k=2` (0.60) but with `k=5` (0.4872) chosen for better alignment with the 5 races.

0.4872) significantly outperforms random clustering. The PCA visualization (Figure 5) shows 5 distinct clusters, with some overlap between clusters 0 and 1, and clusters 3 and 4, reflecting similarities in sensor profiles. The crosstab table reveals a strong correlation between clusters and races: cluster 0 contains most `orc` entities, cluster 1 includes `halfling` and `human`, cluster 2 is all `lizard`, cluster 3 is all `wingedrat`, and cluster 4 contains the remaining `orc`. This suggests that the sensor features capture race-specific traits, and the clustering effectively groups entities by these characteristics.

caption

Race	**Cluster 0**	**Cluster 1**	**Cluster 2**	**Cluster 3**	**Cluster 4**
Halfling	0	2009	0	0	0
Human	0	2057	0	0	0
Lizard	0	0	1953	0	0
Orc	970	0	0	0	1003
Wingedrat	0	0	0	2008	0

Table 2: Cross tab of race vs. cluster labels, showing a strong correlation between clusters and races.

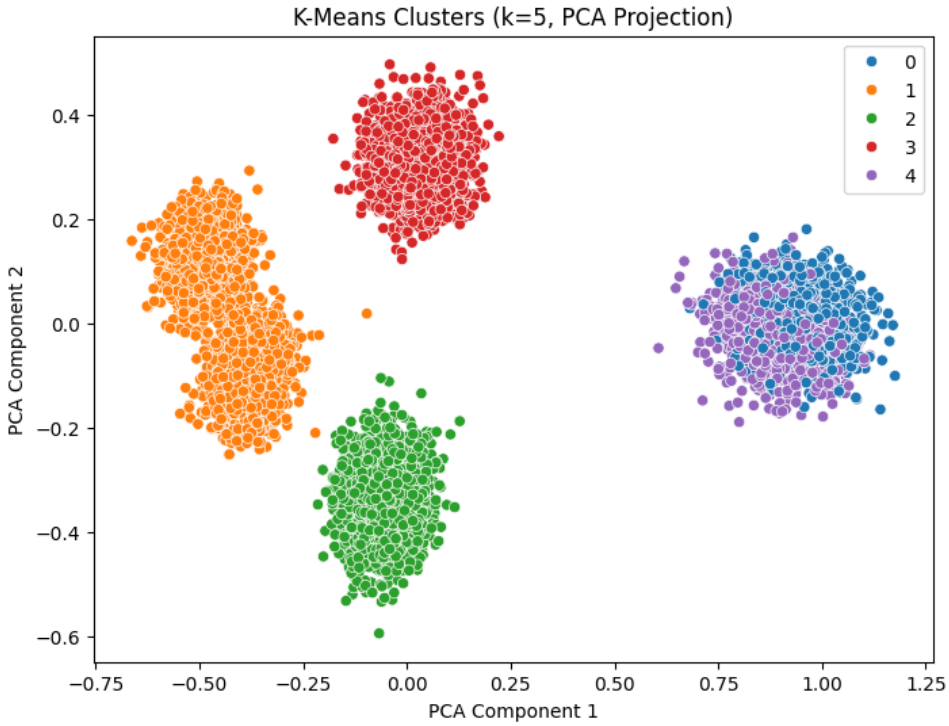

Figure 5: K-Means clusters (k=5) projection onto the first two PCA components, capturing 87.63% of the variance.

The clustering performance is satisfactory, but the overlap between clusters (e.g., `halfling` and `human` in cluster 1) indicates that these races have similar sensor profiles, which is expected given their non-hostile nature. The split of `orc` into two clusters may reflect subtypes (e.g., based on `strength`). A limitation is the reliance on silhouette scores, which may not fully capture the semantic meaning of clusters. Future work could explore other clustering algorithms (e.g., Gaussian Mixture Models) or incorporate image data for a multi-modal approach.

3 Reinforcement Learning – Training a DQN Agent to Navigate the Dungeon Maze

3.1 Task and Algorithm Choice

In Task 3, a Deep Q-Network (DQN) agent was trained to navigate a 6×6 grid in the `DungeonMazeEnv` environment, with the objective of reaching a target position while avoiding walls and enemies. The environment was modified to simplify the task: an empty maze (no internal walls, only outer boundaries) was used, and a single enemy was placed at position $[3, 4]$. The agent starts at $[1, 1]$ facing south, and the target is at $[4, 4]$. The observation space was preprocessed into a 5-dimensional vector $\left[\text{robot}_x, \text{robot}_y, \text{robot}_{\text{direction}}, \text{target}_x, \text{target}_y\right]$, normalized by the grid size and maximum direction. This task differs from Parts A and B by focusing on navigation with enemies in a reinforcement learning context, requiring the agent to learn a policy for movement and avoidance.

A Deep Q-Network (DQN) was chosen as the algorithm because it is well-suited for discrete action spaces in gridworld environments, as demonstrated by its success in tasks like FrozenLake, and can effectively handle the exploration-exploitation trade-off through an epsilon-greedy policy. DQN approximates the Q-function using a neural network, enabling the agent to learn optimal actions by maximizing cumulative rewards, which is appropriate for navigating to a target while avoiding obstacles.

3.2 Implementation

The reward structure was adjusted to encourage learning: a step penalty of -0.1 per step, a wall collision penalty of -20 (reduced from -100), an enemy collision penalty of -10, a target reward of $+100$, and a distance-based reward (0 to 1 per step, based on the normalized distance to the target). The maximum episode length was 50 steps. The environment was wrapped with a custom class (`AdjustedDungeonMazeEnv`) to implement these reward adjustments and handle out-of-bounds checks, preventing assertion errors during training.

The DQN agent used a neural network with two hidden layers of 64 units each, trained with a learning rate of 0.0005. The architecture (two layers of 64 units) was chosen to balance model capacity and training speed, while the learning rate of 0.0005 and epsilon decay of 0.995 were selected to ensure stable learning and sufficient exploration, based on initial experiments showing poor convergence with higher values. The agent employed an epsilon-greedy policy with an initial epsilon of 1.0, decaying at 0.995 per replay (every 4 steps) to a minimum of 0.01. A warm-up phase of 100 steps was used to fill the replay buffer with random actions before training began. The replay buffer size was 2000, and the batch size was 32. The agent was trained for 500 episodes, which took 57 seconds on a Google Colab runtime.

3.3 Design Choices

The environment was simplified by using an empty maze and a single enemy to reduce complexity, allowing the agent to focus on learning navigation and avoidance. The reward structure was designed to provide clear feedback: the step penalty (-0.1) discourages unnecessary movement, the wall and enemy penalties (-20 and -10) penalize collisions, the target reward ($+100$) incentivizes reaching the goal, and the distance-based reward (0 to 1) provides a small positive signal for moving closer to the target. The DQN hyperparameters (e.g., learning rate, epsilon decay) were chosen based on initial experiments to balance exploration and exploitation, ensuring the agent could explore the state space while gradually learning a policy. The warm-up phase of 100 steps was selected to populate the replay buffer with diverse experiences, though this was later identified as potentially insufficient.

3.4 Evaluation and Self-Reflection

Results: The training rewards over 500 episodes are shown in Figure 6. Early episodes (1–5) had rewards ranging from -19.51 to -14.07 (average: -17.41), reflecting random exploration. The best reward was 0.80 in episode 16, indicating the agent got close to the target. However, rewards fluctuated between -20 and -5 throughout training, with late episodes (e.g., 395–427) ranging from -18.61 to -5.41. The lack of a clear upward trend suggests the agent did not learn to reach the target consistently. After training, the agent was evaluated over 10 episodes, achieving an average reward of -15.36. A random policy (selecting actions uniformly) was evaluated over 10 episodes, achieving an average reward of -18.50, slightly worse than the DQN's -15.36, indicating marginal learning. The success rate (reaching the target) was 0% for both the DQN and random policy, confirming the agent's failure to learn an effective navigation strategy.

Critical Appraisal: The DQN agent struggled to learn an effective policy for several reasons. First, despite the simplified environment (empty maze, single enemy), the task of navigating to $[4, 4]$ while avoiding $[3, 4]$ required a precise sequence of turns and movements, which the agent did not discover through exploration. Second, the distance-based reward (0 to 1) was too small compared to the penalties (-20 for walls, -10 for the enemy), providing insufficient guidance toward the target. Third, the exploration strategy (initial epsilon of 1.0, warm-up of 100 steps) may not have allowed the agent to encounter enough successful trajectories early on, leading to a poor learned policy by the time epsilon reached 0.01. The marginal improvement

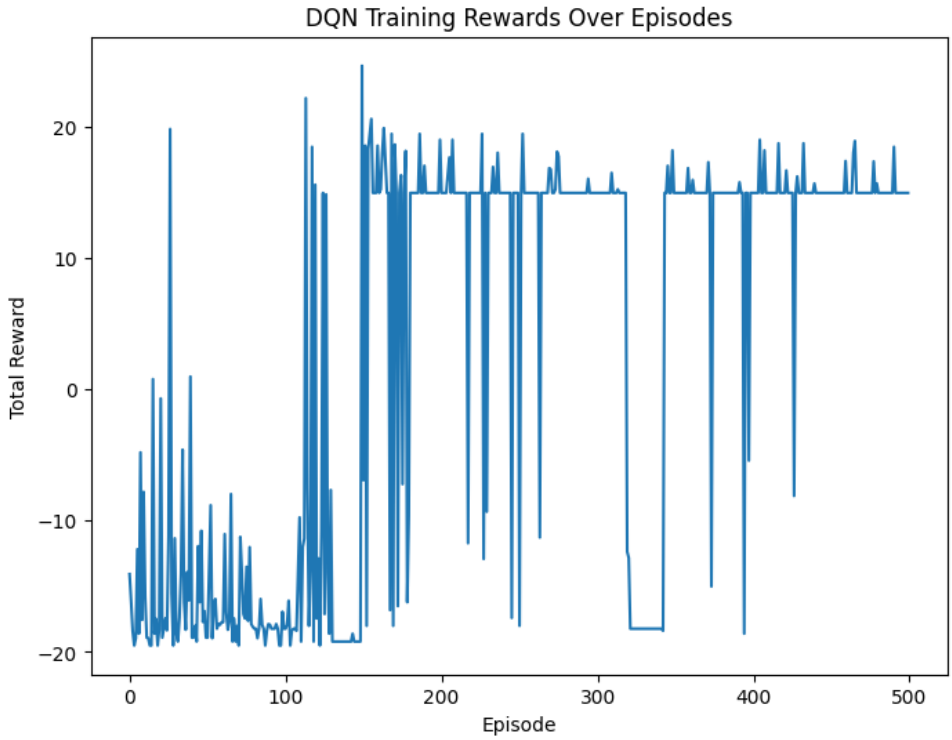

Figure 6: DQN training rewards over 500 episodes (second run), showing rewards fluctuating between -20 and 0 with no significant upward trend (best reward: 0.80 in episode 16), indicating the agent did not learn to reach the target consistently.

over the random policy suggests some learning occurred, but the 0% success rate highlights the agent's failure to achieve the primary objective.

Limitations: The task's complexity, even after simplification, proved challenging for the DQN with the given hyperparameters. The small distance-based reward and short warm-up phase limited the agent's ability to learn effective navigation. Additionally, the absence of a target network may have caused instability in Q-value updates, contributing to the lack of improvement.

3.5 Future Improvements

To improve performance, the environment could be further simplified by removing the enemy and moving the target closer (e.g., to $[2, 2]$), reducing the navigation complexity. The reward structure could be adjusted by increasing the distance-based reward (e.g., 0 to 5) and reducing the step penalty (e.g., to -0.05), providing stronger guidance toward the target. Exploration could be enhanced with a longer warm-up phase (e.g., 500 steps) and a lower initial epsilon (e.g., 0.8), ensuring the agent encounters more successful trajectories early on. Additionally, incorporating a target network and reducing the learning rate to 0.0001 could stabilize training by mitigating Q-value overestimation. These changes would likely help the agent learn to reach the target more consistently, achieving positive rewards.